

C Practice Projects Roadmap

You already understand: - Variables - Functions - Switch statements - Basic pointers - Swapping via pointers The next step is learning how C handles: memory, arrays, strings, structs, file handling, and dynamic allocation. The projects below gradually increase in difficulty. The goal is not only to finish them, but to understand WHY C behaves the way it does.

Project 1 — Number Statistics Analyzer

Practice arrays, loops, pointers, and functions.

Requirements:

- User enters amount of numbers
- Store numbers in an array
- Calculate highest, lowest, average, and total sum

Restrictions:

- Use functions
- At least one function must use pointer arithmetic instead of array indexing

Stretch Goal:

- Implement ascending and descending sorting manually.

Project 2 — String Tools Program

Practice strings, character arrays, and char pointers.

Features:

1. Count characters
2. Reverse string
3. Convert to uppercase
4. Count vowels

Restrictions:

- Do not use `strlen()`, `strrev()`, or similar convenience functions
- Walk through the string manually.

Important concept:

Strings in C are arrays of characters ending with `'\0'`.

Project 3 — Student Grade System

Practice structs, arrays of structs, and functions.

Create a Student struct containing:

- Name
- Age
- Grade

Features:

- Add students
- Show all students
- Find highest grade
- Calculate average grade

Stretch Goal:

- Sort students by grade.

Project 4 — Pointer Maze

Practice deeper pointer understanding.

Requirements:

- Create an integer array
- Print values using indexing
- Print values using pointers
- Print memory addresses
- Traverse arrays using ONLY pointer arithmetic

Questions to investigate:

- Why does `ptr + 1` move several bytes?
- Why can arrays sometimes behave like pointers?

Project 5 — Dynamic Memory Manager

Practice `malloc`, `calloc`, `free`, and heap memory.

Requirements:

- User chooses amount of numbers
- Dynamically allocate memory
- Fill array
- Print numbers
- Calculate average
- Free memory afterward

Key Concept:

- Understand the difference between stack memory and heap memory.

Project 6 — File-Based Notes App

Practice file input/output and persistence.

Features:

1. Add note
2. View notes
3. Clear notes

Use:

- `fopen`
- `fprintf`
- `fgets`
- `fclose`

Stretch Goal:

- Add timestamps to notes.

Project 7 — Mini Banking System

Combine everything learned so far.

Features:

- Create account
- Deposit
- Withdraw
- Save to file
- Load from file

Use:

- Structs
- Arrays
- Pointers
- File handling
- Functions

Final Notes

Important Advice

Do not just aim to finish the projects.

While coding, constantly ask yourself:

- Where is this variable stored?
- Am I passing a copy or an address?
- Who owns this memory?
- What happens if this pointer becomes invalid?

That mindset is what builds real understanding in C.

Recommended Learning Order
1. Statistics Analyzer
2. String Tools
3. Student Grade System
4. Pointer Maze
5. Dynamic Memory Manager
6. File Notes App
7. Banking System